Course Project: Program Testing and Coverage Analysis

In this project you will analyze an instructor-supplied C program (source code provided: bzip2.c). Your job is to apply greybox fuzzing, measure and compare coverage, and map coverage onto the programs' CFGs. You will also recommend how the program's testability and coverage could be improved if you were the developer.

Learning Objectives

- Generate CFGs and functions call graph.
- Use a Fuzzing engine (AFL++) on source code.
- Produce and interpret source-level coverage reports.
- Map coverage to CFG nodes and analyze fuzzing performance.
- Recommend developer-side improvements to increase coverage and testability.

Project Format

- **Team Size:** Work in teams of 3 students**.**
- **Team Formation:** Students should form their own teams as soon as possible.
- **Collaboration Rules:**
    - All team members are expected to contribute substantially to all aspects of the project.
    - Each member should be able to explain the methodology, results, and analysis during presentations or questions.
- **Resources:** Teams are allowed (in fact, encouraged) to use any online resources, including documentation, tutorials, research papers, and AI LLMs, for guidance and reference.
- **Original Work:** While external resources are allowed, the submitted work (analysis, visualizations, and reports) must be original and written by the team. Any direct copying from external sources must be clearly cited.
- **Deliverables:** Teams will submit one collective set of deliverables per team. All members will receive the same grade unless the instructor determines unequal participation.

Project Tasks

1. Call Graph and Control Flow Graph Generation (15 pts)

- Generate the program's call graph.
- Generate Control Flow Graphs (CFGs) for the following functions:
    1. uncompressStream (Line: 5416).
    2. BZ2_bzWrite (Line: 4366).
- Suggested tool: LLVM (opt -dot-cfg) or other static analysis tools.
- Research online how to generate CFGs and call graphs for C programs using LLVM.
- Prepare diagrams in readable formats (DOT, PNG, PDF) with line numbers and function labels.

2. Greybox Fuzzing Campaign (20 pts)

- Build the programs in Linux (Use VirtualBox or WSL).
- You should test two functionalities in the program:
    1. Compressing a file.
    2. Decompressing a file.
- Run AFL++ **for at least 24 hours** twice, once for each functionality.
    1. Read the official AFL++ documentation to understand setup, instrumentation, and fuzzing workflow.
    2. Use any online resources (tutorials, examples, videos) to learn how to create seeds, and run campaigns.
    3. Check out fuzzing exercises for practice: Fuzzing101 Exercise 1.
- Collect the full fuzzing artifact tree, including:
    1. All directories (queue, crashes, hangs).
    2. fuzzer_stats and progress files.
    3. Initial seed corpus (initial input files).
    4. Command lines used for the run.
- ***Take screenshots of the final AFL++ screen before shutting down.***
- During the demo/presentation, you will be asked to show the artifact pack and rerun at least one testcase.

3. Source-Based Coverage (15 pts)

- After completing the fuzzing campaigns, collect source-based coverage using a tool such as: **afl-cov.**
- Collect the following to include in your final report for each of the two campaigns:
    - Summary table with coverage metrics (lines/functions covered, % coverage).
    - Pictures/screenshots of the generated coverage report from afl-cov of the two functions specified in task 1.
- Research online how to use afl-cov to generate source-based coverage.

4. Coverage Mapping and Analysis (20 pts)

- Map the coverage data from source-based coverage (Task 3) onto the CFGs and call graphs (Task 1).
- Annotate the two CFGs node/edge to indicate whether it was covered or uncovered.
- Annotate the call graph to show which functions were executed by the fuzzer.
- Include visualizations with clear color coding of coverage per function and CFG.

5. Final Report (20 pts)

Each team will compile a final report including the following sections and information. This report will be the main deliverable for the project, and all other data collected in Tasks 1–4 will be included within this report. The report should be clear, organized, and include all relevant figures, tables, and analysis.

1. Program Analysis [2 pages]
Briefly describe the program: what it does and how it works.

- Include the call graph generated in Task 1.
- Highlight the two functions specified in Task 1 in the call graph.

2. Function Analysis [4 pages]
Describe the functionality of the two selected functions.

- Explain exactly what each function does (inputs, outputs, key computations).
- Include the CFGs of these functions.

3. Fuzzing Campaigns [2 pages]
For each of the two fuzzing campaigns, describe your fuzzing setups and execution**:**

- Duration (must run at least 24 hours).
- ***Include screenshots of the final AFL screen before shutting down.***
- Seed files used.
- Key informational resources consulted to learn AFL++ (docs, tutorials, exercises, ChatGPT).
- Number of test cases generated.
- Any crashes or hangs found?
- Fuzzing configurations (e.g., AFL options, timeouts).
- Program configurations during fuzzing (command-line flags).
- Machine specifications (CPU, RAM) and OS.

4. Source-Based Coverage [2 pages]
Discuss source-based coverage of the fuzzing campaign:

- Include the summary tables collected in Task 3
- Analyze coverage per function as mapped in Task 4
- How much of each specified function was covered? How much was not?
- Include color-coded visualizations of function CFG coverage

5. Discussion [5 pages]

Use this section to reflect on your fuzzing results, coverage outcomes, and overall testing experience. Your discussion should address the following subsections:

1. Coverage Patterns [3 paragraphs]
   - Which parts of the program were well-covered by fuzzing? Why?
   - Which parts were poorly covered or missed entirely?
   - How does program structure (loops, conditionals, parsing logic) affect which areas were reached?

2. Function-Level Coverage [2 paragraphs]
   - For the two functions you analyzed, which parts (branches, loops, error paths) were covered or uncovered?
   - What might explain the uncovered portions?

3. Crash analysis [3 paragraphs]
   - For the found crashes, briefly analyze the bug behavior.
   - In what functionality/function/Line-of-code is the bug?
   - What kind of bug is it (i.e. memory-bug?)?
   - How to fix it?

4. Source-Based vs Graph Coverage [1 paragraph]
   - You used source-based coverage (afl-cov) and mapped it to your CFGs.
     - How does source-based coverage compare to graph coverage?
     - Which one provides more useful insight into untested paths?
     - Are there situations where graph coverage could highlight execution paths that source-based metrics overlook?

5. Program Testability and Coverage Barriers [3 paragraphs]
   - Was the program easy or difficult to fuzz? Why?
   - What characteristics (input format, control flow, data dependencies) helped or hindered coverage?
   - If you were the developer, what changes could improve testability and coverage? (e.g., simplifying input parsing, modular design, better error handling).

6. Tool Effectiveness and Limitations [1 paragraph]
   - How effective was AFL++ at finding diverse inputs or revealing bugs?
   - Did you encounter limitations (e.g., hard-to-reach code, complex input constraints, long execution times)?
   - What strategies could improve fuzzing effectiveness in future campaigns?

7. Seeds and Fuzzing Variability [1 paragraph]
    - Run two short (1-hour) fuzzing campaigns with different initial seeds.
        - Run one campaign with a single file as a seed (say containing one character).
        - Run the second campaign with a collection of valid bzip-compressed files.
        - Run both campaigns for the decompression functionality.
        - Which of the two campaigns had better coverage?
        - Were any new paths or crashes discovered?
        - What does this tell you about the impact of seed selection in fuzzing?

8. Reflections and Lessons Learned [2 paragraphs]
    - What surprised you about fuzzing or coverage behavior?
    - What would you do differently if you repeated this experiment?
    - How could the insights from this project apply to real-world software testing or development?

6. Team Demo (10 pts)

- No slides or formal presentation required.
- Each team should bring their laptop and connect it to the classroom display (HDMI).
- During the demo (5 minutes), the team must:
    1. Show the fuzzing artifact pack collected in Task 2.
    2. Run the fuzzer briefly to demonstrate it is functional.
    3. Execute at least one test case discovered by the fuzzer.
    4. Open and show the afl-cov HTML report generated from Task 3.
- Teams should be ready to answer brief questions about their fuzzing setup, coverage, and CFG analysis.

Timeline

- **Team formation:** by Week 11 (no deliverable).
- **Preliminary CFGs & call graphs:** Week 12 (no deliverable).
- **Fuzzing campaign complete:** Week 13 (no deliverable).
- **Demo:** Week 15 (May. 6th, in-class).
- **Final report:** Last day of classes (May. 10th).